WEEK 1 · LESSON 6 OF 24 · HANDOUT

Week 1 Project: Train and Study Your Own Language Model

Study time: **about 90 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=kCc8FmEb1nY — open the lesson online to watch inside the course.

🎓 **Go deeper:** Andrej Karpathy: Let's reproduce GPT-2 (124M) — https://www.youtube.com/watch?v=l8pRSuU81PU

🎓 **Go deeper:** Andrej Karpathy: The spelled-out intro to language modeling — building makemore — https://www.youtube.com/watch?v=PaCmpygFfXo

Your goal in this lesson: Train a small GPT on a text you choose, measure it with validation loss, explore sampling settings, and explain its strengths and failures — including hallucination — in a short report.

The brief

A teacher wants a demonstration for a school club: "How does ChatGPT write?" Your job is to train a small language model, show how it improves, show what sampling does, and explain honestly what it gets wrong.

Part 1 — Choose your text

Use the market sentences, or replace `text` with your own: proverbs, simple stories or poems you wrote yourself, or public-domain text. Aim for at least 50,000 characters of fairly consistent style. Do not use private messages or copyrighted books.

```
import random

random.seed(0)
names = ["Aline", "Eric", "Grace", "Jean", "Diane", "Patrick", "Claudine", "Samuel", "Ange",
"Didier"]
places = ["Kigali", "Huye", "Musanze", "Rubavu", "Nyagatare", "Rusizi", "Muhanga"]
goods = ["tomatoes", "beans", "bananas", "avocados", "maize", "potatoes", "milk", "coffee",
"rice", "cassava"]
verbs = ["sells", "buys", "carries", "cooks", "packs", "counts"]
times = ["in the morning", "at noon", "in the evening", "on Saturday", "after the rain",
"before school"]
moods = ["happy", "tired", "busy", "proud", "calm", "hungry"]
templates = [
    "{n} {v} {g} in {p} {t}.",
    "In {p}, {n} {v} {g} {t}.",
    "{n} is {f} because the {g} are fresh.",
    "{t}, {n} goes to the market in {p}.",
    "{n} pays for the {g} with mobile money.",
    "The price of {g} in {p} is low {t}.",
    "{n} tells {m} that the {g} from {p} are the best.",
    "{n} and {m} share {g} {t}.",
]

def sentence():
    n, m = random.sample(names, 2)
    s = random.choice(templates).format(n=n, m=m, v=random.choice(verbs),
g=random.choice(goods),
                                        p=random.choice(places), t=random.choice(times),
f=random.choice(moods))
    return s[0].upper() + s[1:]

text = "\n".join(sentence() for _ in range(4000))
print(len(text), "characters")
print(text[:250])
```

Part 2 — Build the model

```
import torch
import torch.nn as nn
import torch.nn.functional as F

device = "cuda" if torch.cuda.is_available() else "cpu"
chars = sorted(set(text))
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for c, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: "".join(itos[i] for i in ids)

data = torch.tensor(encode(text), dtype=torch.long)
split = int(0.9 * len(data))
train_data, val_data = data[:split], data[split:]

BLOCK = 64          # how many characters the model can look back at (its context window)

def get_batch(source, batch_size=32):
    starts = torch.randint(len(source) - BLOCK - 1, (batch_size,))
    x = torch.stack([source[s:s + BLOCK] for s in starts])
    y = torch.stack([source[s + 1:s + BLOCK + 1] for s in starts])   # the next character at
every position
    return x.to(device), y.to(device)

class Block(nn.Module):
    # One transformer block: masked self-attention + feed-forward, with residuals.
    def __init__(self, d, heads):
        super().__init__()
        self.ln1, self.ln2 = nn.LayerNorm(d), nn.LayerNorm(d)
        self.attn = nn.MultiheadAttention(d, heads, batch_first=True)
        self.ff = nn.Sequential(nn.Linear(d, 4 * d), nn.GELU(), nn.Linear(4 * d, d))
        mask = torch.triu(torch.ones(BLOCK, BLOCK, dtype=torch.bool), diagonal=1)
        self.register_buffer("mask", mask)              # True = may NOT look (the future)

    def forward(self, x):
        T = x.shape[1]
        h = self.ln1(x)
        a, _ = self.attn(h, h, h, attn_mask=self.mask[:T, :T], need_weights=False)
        x = x + a
        return x + self.ff(self.ln2(x))

class TinyGPT(nn.Module):
    def __init__(self, vocab, d=96, heads=4, layers=3):
        super().__init__()
        self.tok = nn.Embedding(vocab, d)
        self.pos = nn.Embedding(BLOCK, d)
        self.blocks = nn.Sequential(*[Block(d, heads) for _ in range(layers)])
        self.ln = nn.LayerNorm(d)
        self.head = nn.Linear(d, vocab)

    def forward(self, idx):
        T = idx.shape[1]
        x = self.tok(idx) + self.pos(torch.arange(T, device=idx.device))
        return self.head(self.ln(self.blocks(x)))       # logits: a score for every next
character

    @torch.no_grad()
    def generate(self, idx, n_new, temperature=1.0, top_k=None):
        for _ in range(n_new):
            logits = self(idx[:, -BLOCK:])[:, -1, :] / temperature
```

```
            if top_k is not None:
                cutoff = torch.topk(logits, top_k).values[:, [-1]]
                logits[logits < cutoff] = -float("inf")
            next_id = torch.multinomial(F.softmax(logits, dim=-1), num_samples=1)
            idx = torch.cat([idx, next_id], dim=1)
        return idx

@torch.no_grad()
def estimate_loss(model, steps=20):
    model.eval()
    out = {}
    for name, source in [("train", train_data), ("val", val_data)]:
        losses = []
        for _ in range(steps):
            x, y = get_batch(source)
            logits = model(x)
            losses.append(F.cross_entropy(logits.reshape(-1, logits.shape[-1]),
y.reshape(-1)).item())
        out[name] = sum(losses) / len(losses)
    model.train()
    return out

def train_gpt(steps=1500, seed=0):
    torch.manual_seed(seed)
    model = TinyGPT(len(chars)).to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
    for step in range(steps + 1):
        if step % 500 == 0:
            l = estimate_loss(model)
            print(f"step {step:4d}: train loss {l['train']:.3f}, validation loss
{l['val']:.3f}")
        if step == steps:
            break
        x, y = get_batch(train_data)
        logits = model(x)
        loss = F.cross_entropy(logits.reshape(-1, logits.shape[-1]), y.reshape(-1))
        opt.zero_grad()
        loss.backward()
        opt.step()
    return model

print("vocabulary:", len(chars), "characters")
```

Part 3 — Train and measure

```
model = train_gpt(steps=1500)
final = estimate_loss(model, steps=50)
print(f"final: train {final['train']:.3f}, validation {final['val']:.3f}")
```

Validation loss is measured on the last 10% of the text, which the model never trains on. If validation loss rises while training loss falls, the model is memorising (overfitting) — exactly as in earlier courses.

Perplexity = e^loss is a common way to report it: roughly "how many characters the model is choosing between". A loss of 1.0 means a perplexity of about 2.7.

```
import math
print("validation perplexity:", round(math.exp(final["val"]), 2))
```

Part 4 — Explore sampling

```
for T in [0.3, 0.8, 1.3]:
    torch.manual_seed(2)
    out = model.generate(torch.tensor([encode("In ")], device=device), 150, temperature=T)
    print(f"--- temperature {T} ---")
    print(decode(out[0].tolist()))
```

Record what changes: repetition at low temperature, spelling mistakes and odd words at high temperature.

Part 5 — Test for "hallucination"

```
torch.manual_seed(3)
samples = decode(model.generate(torch.tensor([encode("\n")], device=device), 2000,
temperature=0.8)[0].tolist())
generated = [s.strip() for s in samples.split("\n") if s.strip().endswith(".")]
training_sentences = set(text.split("\n"))
new = [s for s in generated if s not in training_sentences]
print(f"{len(generated)} complete sentences, {len(new)} never appeared in the training text")
print("examples of new sentences:", new[:3])
```

Many generated sentences are **new combinations**: grammatical, believable, and never in the data. In our market text that is harmless. In a medical or legal assistant, a fluent new "fact" can be dangerous. This is why real systems ground answers in documents (RAG, Week 3) and why people must check important outputs.

Part 6 — Report (one page)

1. **Data:** what text, how many characters, how many different characters.
2. **Model:** layers, width, context length, parameter count.
3. **Training:** loss curve numbers at steps 0, 500, 1000, 1500; final validation perplexity.
4. **Sampling:** three short samples at different temperatures with one sentence each on what you see.
5. **Hallucination:** how many new sentences, two examples, and why this matters for real LLMs.
6. **From tiny to large:** three differences between your model and a commercial LLM.

Marking guide (self-check)

Area	Excellent work
Training	Model trains; train and validation losses reported
Understanding	Causal mask, next-token targets and sampling explained in your own words
Experiment	Temperature comparison with evidence
Critical thinking	Hallucination measured and explained
Communication	A school club member understands the report

Remember this: You trained a language model from zero and saw both its power (fluent patterns) and its core weakness (likely is not the same as true).

Name: _____ Date: _____

Activity

Task 1

Complete Parts 1–6. Challenge: double `BLOCK` to 128 and the width to 128, train again on a GPU, and compare validation loss and samples.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. Why is validation loss measured on text the model never trained on?
   - A. To check it generalises instead of memorising
   - B. To make the loss smaller
   - C. To speed up training
   - D. To change the vocabulary
2. A validation loss of 1.0 corresponds to a perplexity of about…
   - A. 1
   - B. 2.7
   - C. 10
   - D. 100
3. What usually happens at a very low temperature?
   - A. Many spelling mistakes
   - B. Random characters
   - C. No text at all
   - D. Repetitive, predictable text
4. The model writes believable sentences that never appeared in its data. Why does this matter for real LLMs?
   - A. It doesn't matter
   - B. Fluent invented 'facts' can mislead in medical, legal or financial uses
   - C. It proves the data leaked
   - D. It means the model is broken
5. Which text is suitable for the project?
   - A. Friends' private messages
   - B. Passwords
   - C. A copyrighted novel
   - D. Text you wrote or public-domain text
6. What is one reason commercial LLMs write far better than your tiny model?
   - A. Vastly more parameters, data and context
   - B. They use bigrams
   - C. They use no transformer
   - D. They skip training

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.